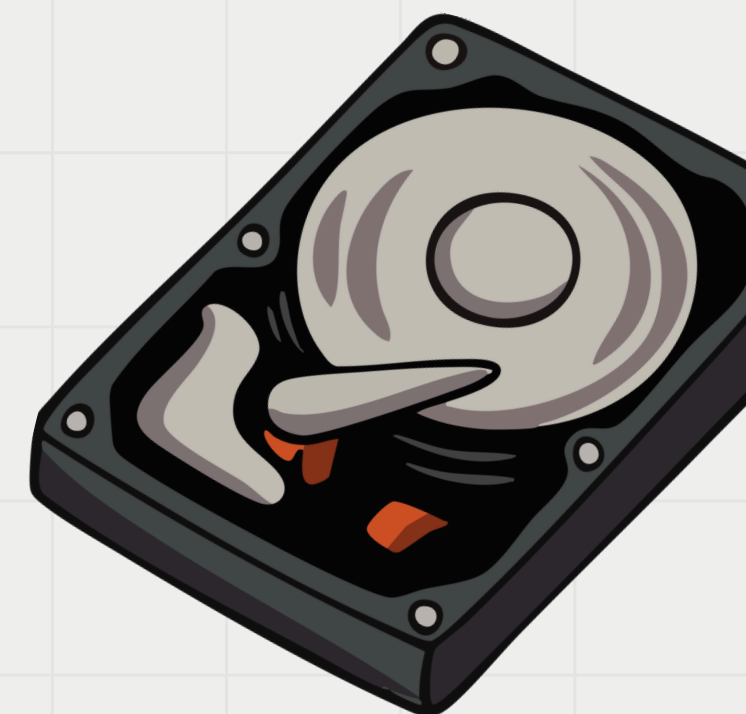
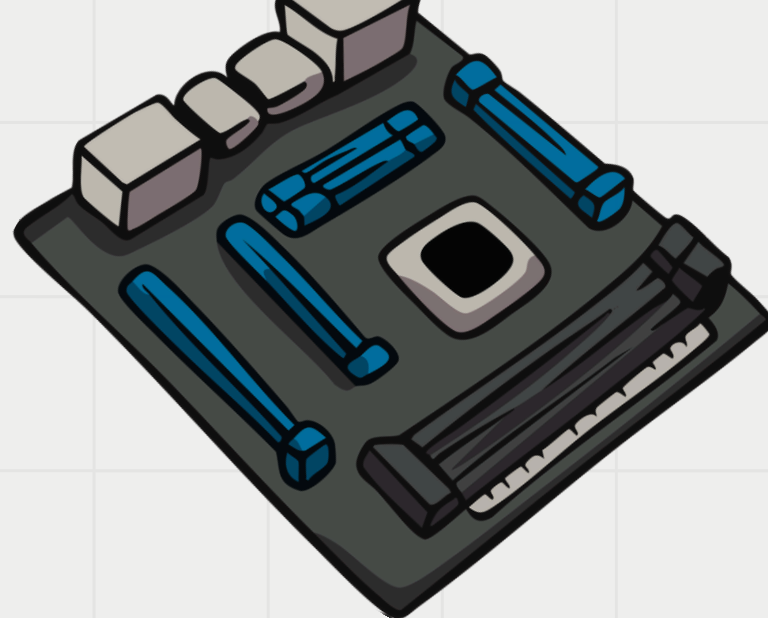
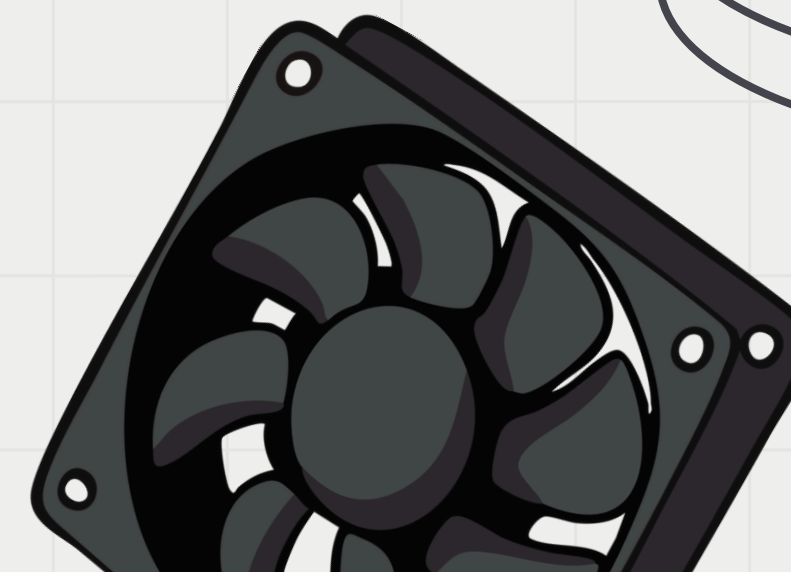
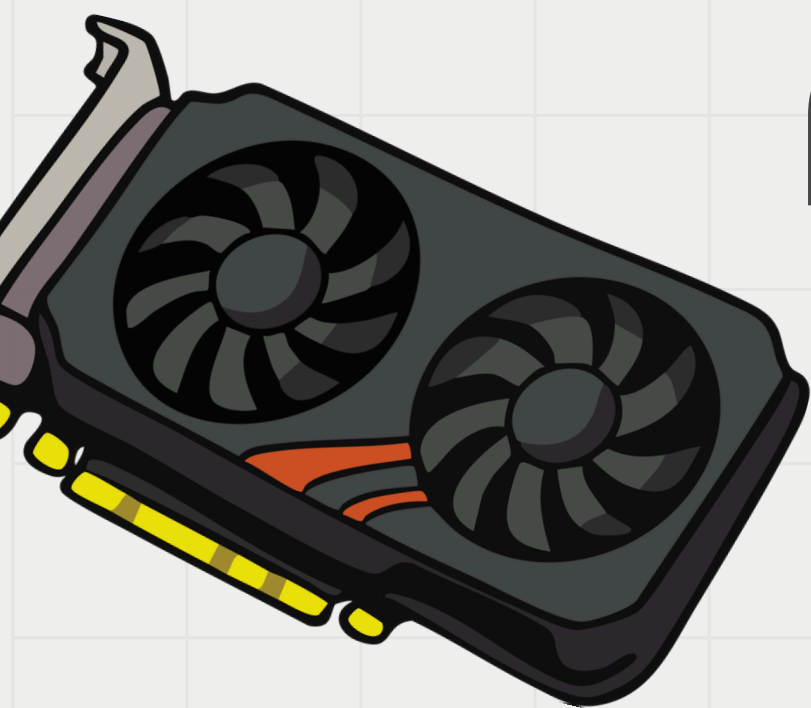
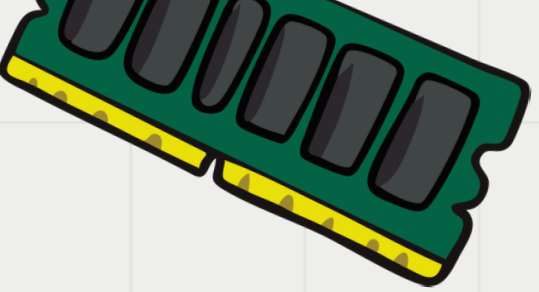




ระบบซื้อขายอุปกรณ์ คอมพิวเตอร์มือสอง





รายชื่อสมาชิก



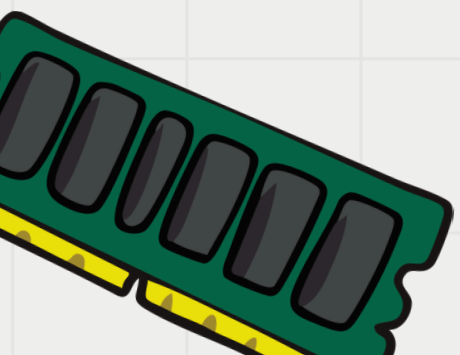
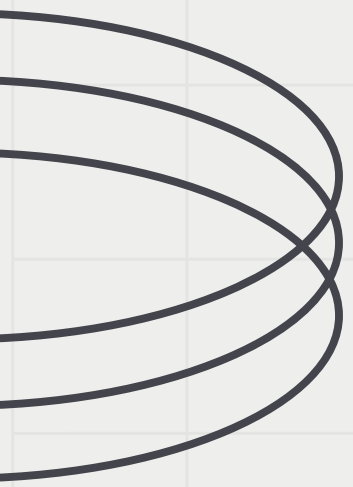
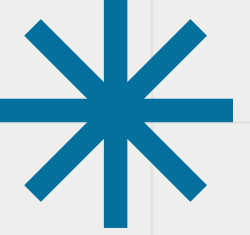
673380038-9 นายณัฐพงศ์ กรรณกิจ

673380040-2 นายธนกร ทองศรี

673380042-8 นายรณันชัย พันธราช

673380048-6 นายปวัฒน์ ปัดทุมมา

673380280-2 นายปิยะพล ตุ่นป่า



วัตถุประสงค์

ระบบซื้อขายอุปกรณ์ คอมพิวเตอร์มือสอง

พัฒนาเว็บระบบซื้อขายอุปกรณ์ คอมพิวเตอร์มือสอง

รองรับการค้นหา สั่งซื้อ ติดตาม รับสินค้า และรีวิว

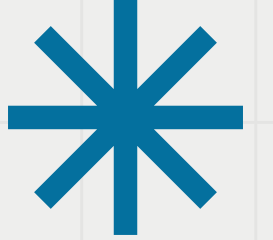
ให้ผู้ขายจัดการร้าน สินค้า คำสั่งซื้อ พร้อมระบบอนุมัติร้านโดยผู้ดูแล

ออกแบบฐานข้อมูลเชื่อมโยงและกำหนดสิทธิ์ตามบทบาท

ทดสอบและแก้ไขข้อบกพร่องเพื่อปรับปรุงระบบ



ขอบเขตของระบบ



ลูกค้า

สมัครสมาชิก เข้าสู่ระบบ ค้นหาสินค้า เพิ่มสินค้าลงตะกร้า เลือกจัดส่ง สั่งซื้อ ตรวจสอบสถานะคำสั่งซื้อ และรีวิว



ผู้ขาย

ขอเปิดร้าน ยืนยันตัวตน เพิ่ม/แก้ไขสินค้า กำหนดราคาและจำนวนสินค้า จัดการคำสั่งซื้อ และบันทึกผู้ขนส่งพร้อมเลขพัสดุ



ผู้ดูแลระบบ

ตรวจสอบคำขอเปิดร้าน อนุมัติหรือปฏิเสธการเปิดร้าน และจัดการหมวดหมู่สินค้า



เทคโนโลยี

Next.js สำหรับส่วนติดต่อผู้ใช้, Spring Boot สำหรับ API, PostgreSQL สำหรับฐานข้อมูล และระบบตรวจสอบสิทธิ์ตามบทบาท

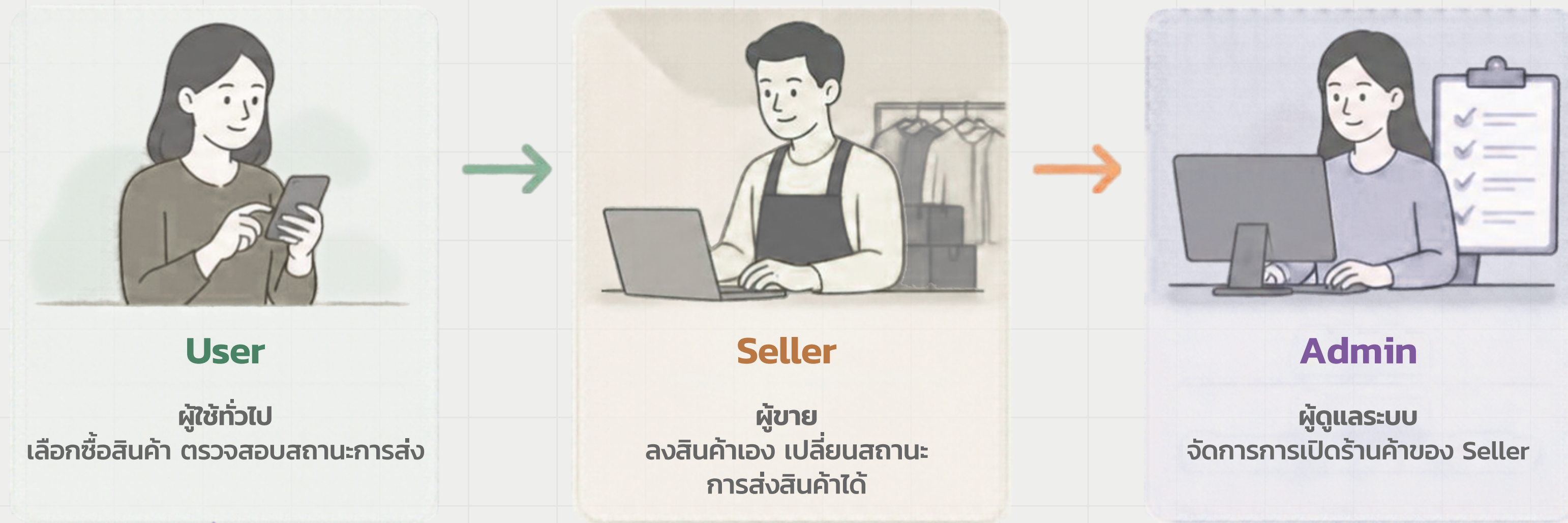


การชำระเงินและการจัดส่ง

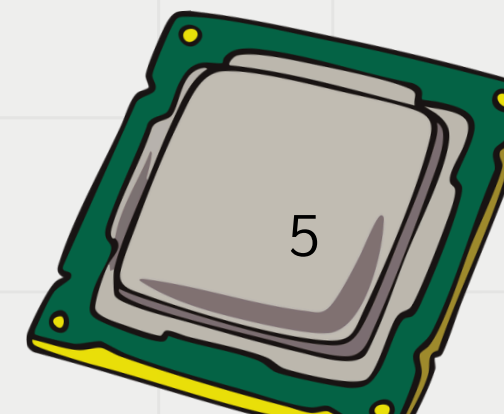
ระบบชำระเงิน จำลองและคำนวณค่าจัดส่ง บันทึกข้อมูลติดตามพัสดุ

ระดับผู้ใช้ในระบบ

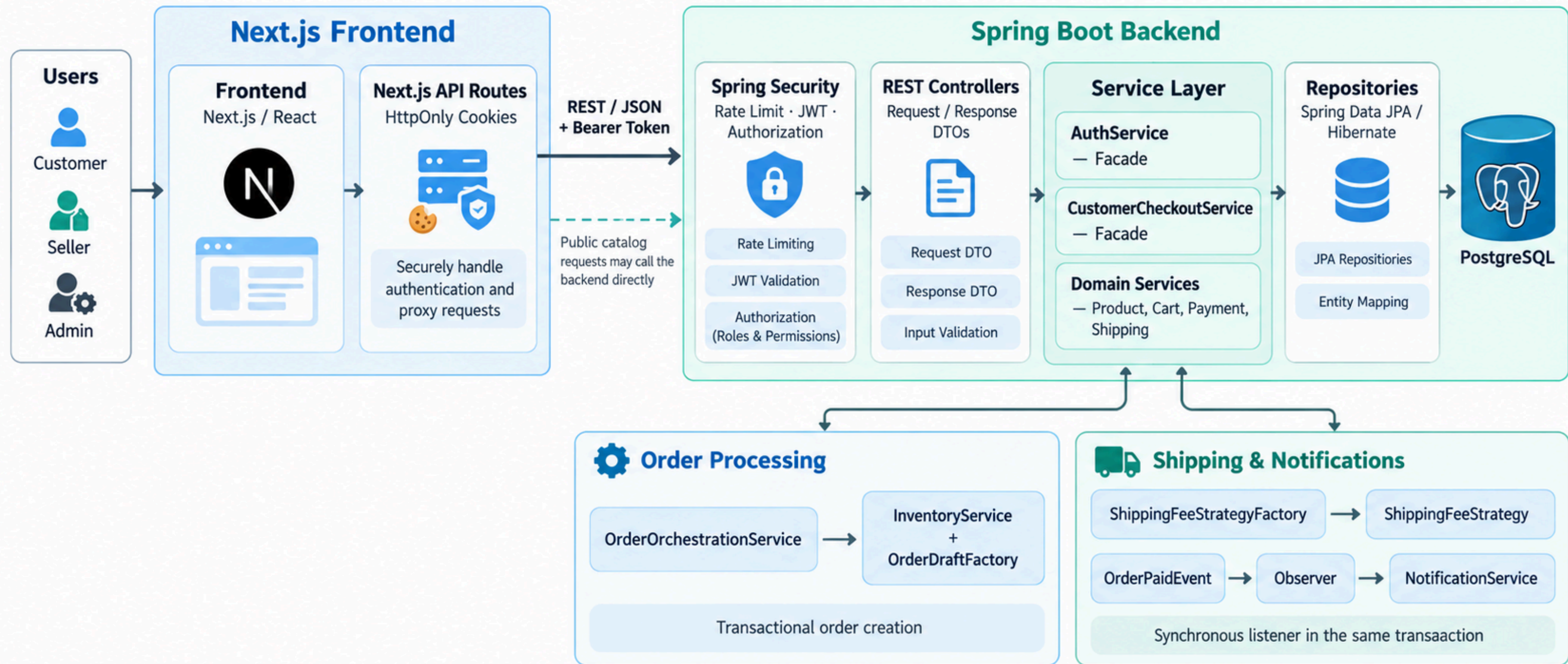
บทบาทและสิทธิ์ของผู้ใช้แต่ละระดับในระบบตลาดสินค้ามือสองแบบหลายร้านค้า



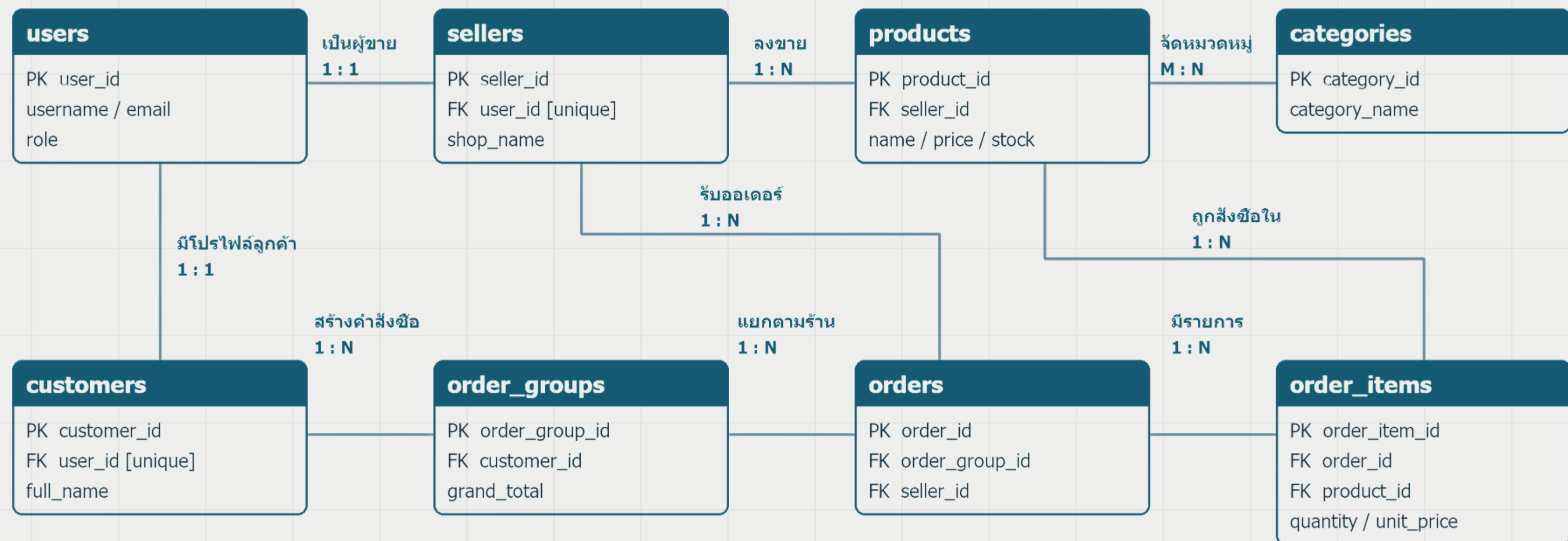
ทุกบัญชีเริ่มต้นในฐานะ **User** ก่อน
สามารถยื่นคำขอเพื่อเป็น **Seller** ได้ภายหลัง

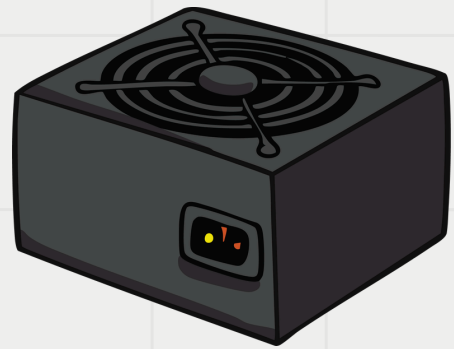


Architecture

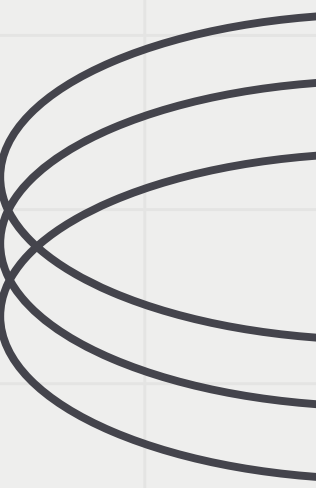


ER Diagram

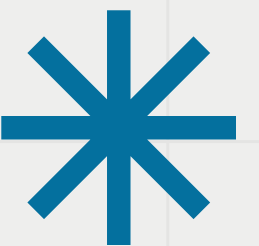
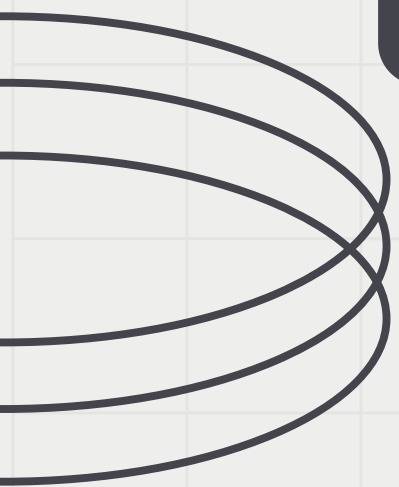




Solid Principle

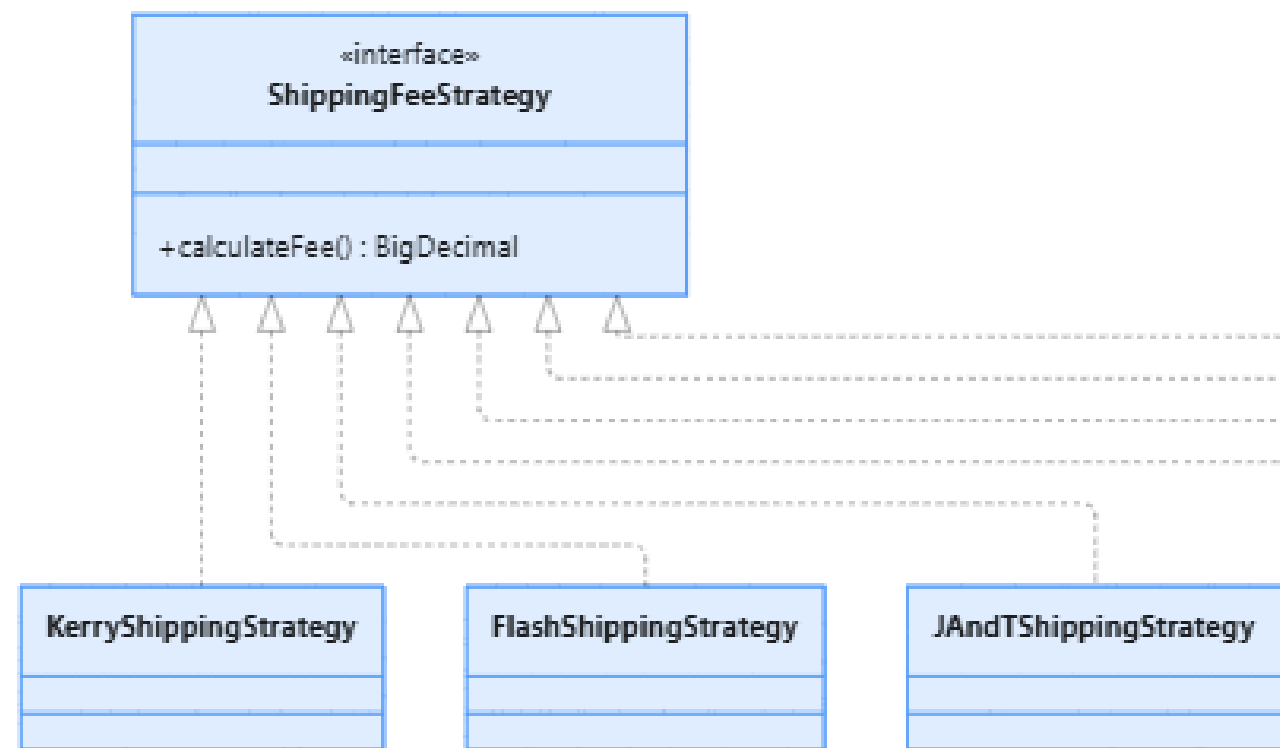


- S Single Responsibility — แต่ละ Service รับผิดชอบงานเดียว เช่น CartService, PaymentService, ShippingService แยกจากกันชัดเจน
- O Open/Closed — เพิ่มบริษัทขนส่งใหม่ได้โดยเพิ่มคลาส ShippingFeeStrategy ใหม่ ไม่ต้องแก้ไขโค้ดเดิม
- L Liskov Substitution — ทุกกลยุทธ์ขนส่ง (Flash, Kerry, EMS, J&T, ไปรษณีย์ไทย, Standard) ใช้แทนกันได้ผ่าน interface เดียวกัน
- I Interface Segregation — แยก Service interface ตามโดเมน เช่น AddressService, CartService, InventoryService ไม่รวมเป็น interface ใหญ่เดียว
- D Dependency Inversion — Controller และ Service พึ่งพา interface (package api) ไม่ใช่คลาส Imp ตรง ๆ
Spring จัด dependency ผ่าน constructor



Design Pattern

Strategy Pattern



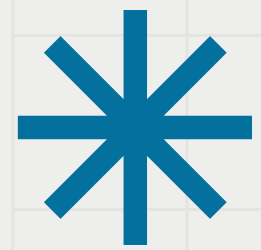
Shipment Service

Shipment Service — Strategy Pattern

แยกวิธีคำนวณค่าส่งของแต่ละบริษัทขนส่งเป็น Strategy โดยใช้ `ShippingFeeStrategy` เป็น interface กลาง

ข้อดีของการออกแบบด้วย Strategy Pattern

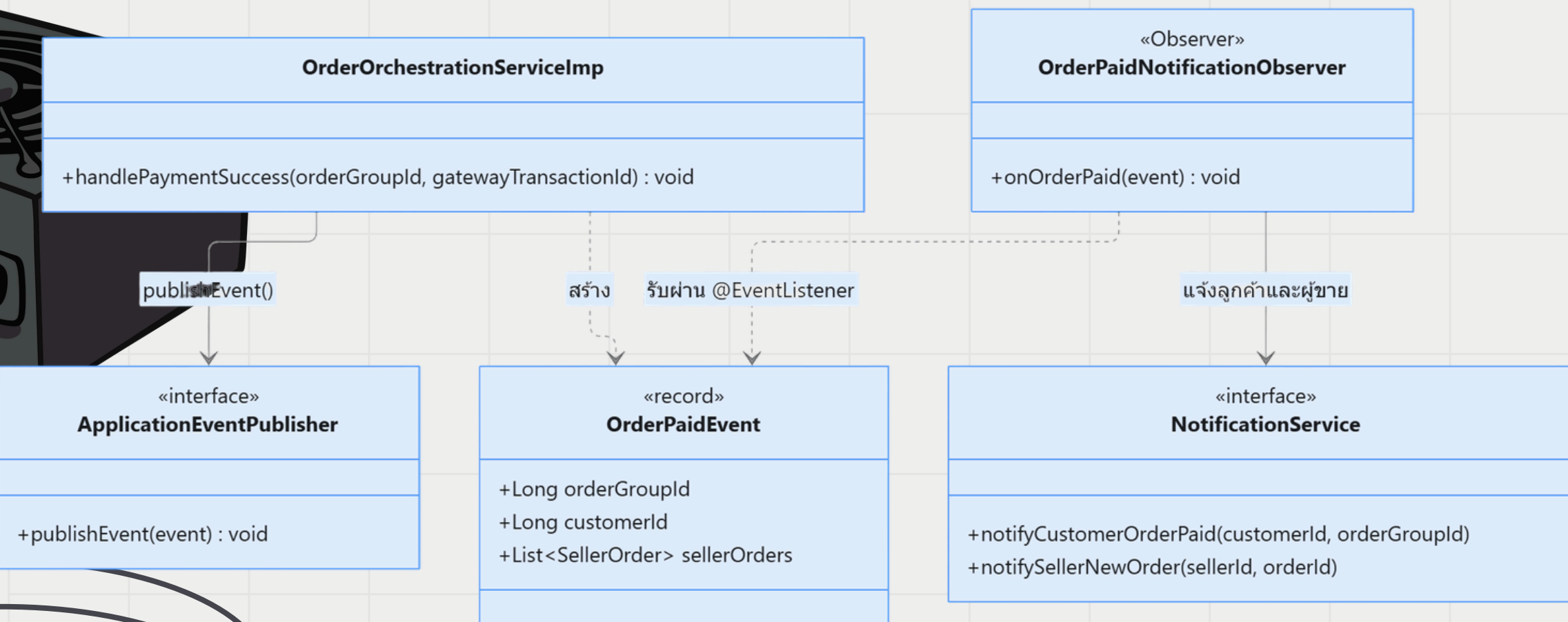
- เพิ่มขนส่งได้ง่าย — สร้าง Strategy ใหม่ได้ง่าย
- แก้ไขได้เฉพาะจุด — ปรับสูตรค่าส่งของแต่ละบริษัทแยกกัน
- รองรับ Open/Closed Principle — เพิ่มวิธีคำนวณใหม่ โดยลดการแก้ไขโค้ดเดิม



Design Pattern

Observer Pattern

เมื่อชำระเงินสำเร็จ ระบบจะ "ประกาศเหตุการณ์" แล้วให้ตัวติดตามไปแจ้งเตือนต่อ โดยไม่ผูกโค้ดกันตรง ๆ



ข้อดี

- แยกตรรกะการแจ้งเตือนออกจากตรรกะการชำระเงิน/คำสั่งซื้อ (loose coupling)
- เพิ่ม Observer ใหม่ (เช่น ส่งอีเมล ส่ง SMS) ได้โดยไม่แก้ Order OrchestrationService
- ทดสอบ Observer แต่ละตัวแยกจาก flow การสั่งซื้อได้

ข้อเสีย

- ตามลำดับการทำงานยากขึ้น เพราะ event ถูกยิงแบบ implicit ไม่เห็น flow ตรง ๆ ในที่เดียว
- debug ยากขึ้นถ้า Observer เกิด error เจียบ ๆ ต้องไล่ดู log เพิ่ม
- มี Observer มากอาจกระทบ performance ของ transaction หลัก

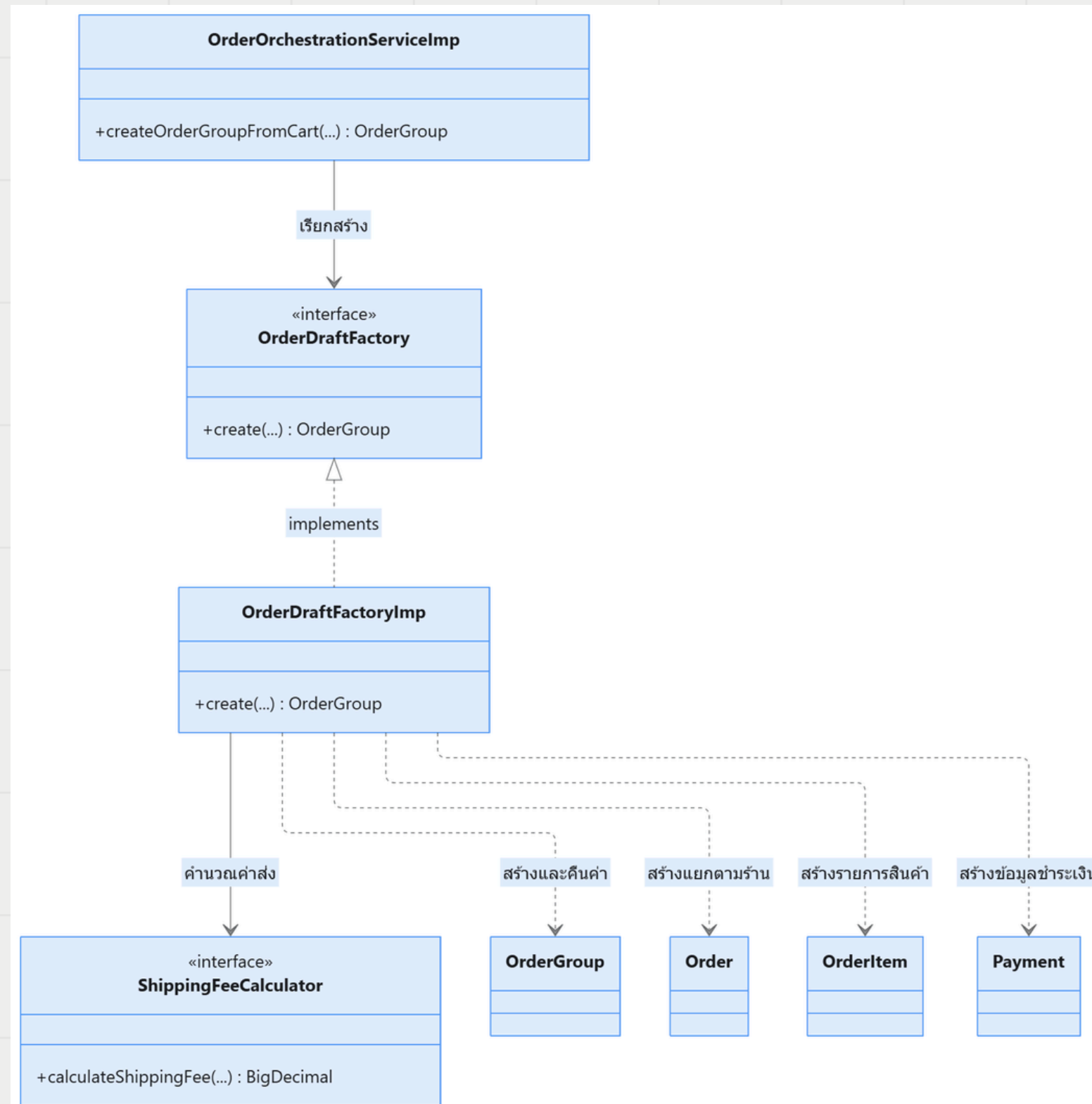




Design Pattern

Factory Pattern

รวม logic การ "สร้างออบเจ็กต์" ที่ซับซ้อนไว้ที่เดียว class ที่เรียกใช้ไม่ต้องรู้ขั้นตอนประกอบข้างใน



ข้อดี

- รวม logic การสร้าง object ที่ซับซ้อนไว้ที่เดียว ไม่กระจายอยู่หลาย service
- ผู้เรียกใช้ไม่ต้องรู้รายละเอียดการประกอบ object เพียงเรียก `create()/getStrategy()`
- แก้ไข/ขยาย logic การสร้างได้ในที่เดียวโดยไม่กระทบผู้เรียกใช้

ข้อเสีย

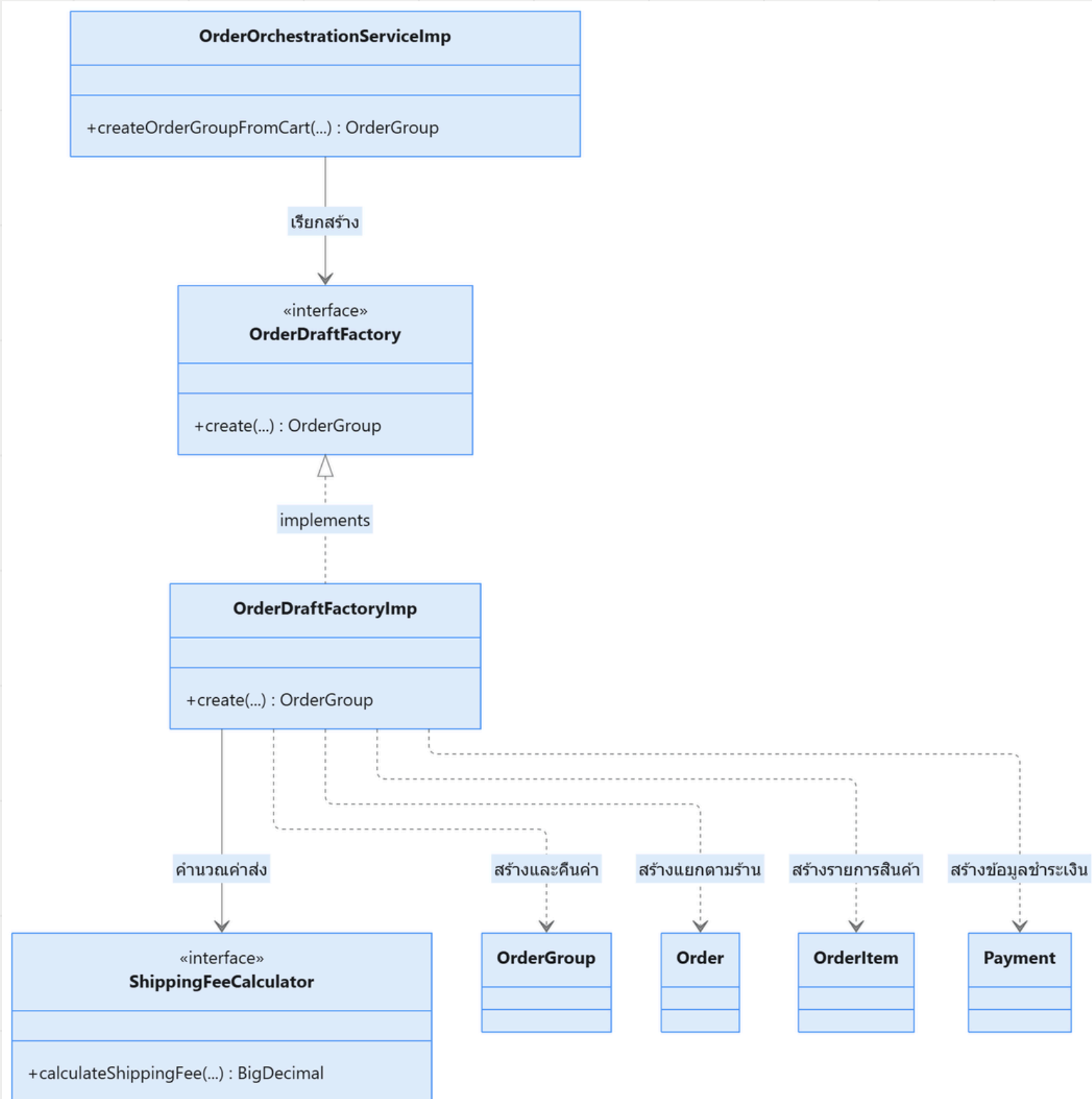
- เพิ่มชั้นของ abstraction อีกชั้น ผู้อ่านโค้ดใหม่ต้องตามเข้าไปดู factory เพิ่ม
- ถ้า factory รับผิดชอบมากเกินไป อาจขัดกับหลัก Single Responsibility
- เพิ่มจำนวนไฟล์/คลาสในโปรเจกต์โดยรวม



Design Pattern

Facade Pattern

CustomerCheckoutService เป็นหน้าด่านเดียวที่ Controller เรียกใช้ ซ่อนความซับซ้อนของหลาย service ย่อยไว้ด้านหลัง

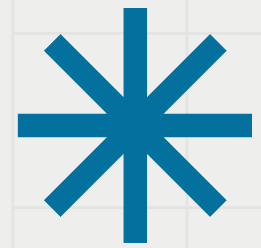


ข้อดี

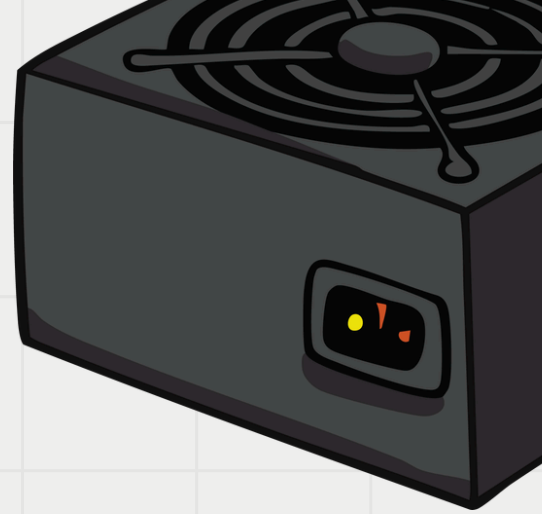
- Controller เรียกใช้งานง่ายผ่าน interface เดียว ไม่ต้องรู้รายละเอียดของ service ย่อย
- ลด coupling ระหว่าง Controller กับ subsystem ภายใน เปลี่ยน implementation ได้โดยไม่กระทบ Controller
- รวม business flow ของการเช็คเอาท์ไว้ที่เดียว อ่านและดูแลง่ายขึ้น

ข้อเสีย

- อาจกลายเป็นคลาส "รู้มากเกินไป" (god object) ถ้าขอบเขตขยายต่อเรื่อย ๆ
- ซ่อน dependency ไว้ภายใน อาจมองไม่เห็นว่ามีที่ subsystem เกี่ยวข้องจริง
- เพิ่ม layer อีกชั้น ทำให้ debug ต้องไล่ผ่าน facade ก่อนถึง service จริง



Security



Authentication — ตรวจสอบตัวตน

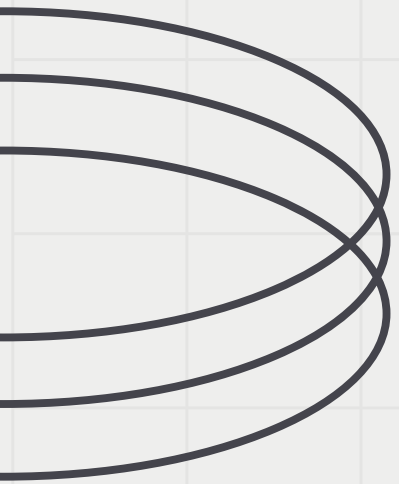
- ตรวจสอบ JWT และ session รวมถึงวันหมดอายุและสถานะบัญชี
- กำหนดตัวตนและ Role ใน Spring Security
- ไม่ผ่านการยืนยันตัวตนใน API ที่ป้องกัน → 401 Unauthorized

Admin Authorization

- ผู้ใช้ที่เข้าสู่ระบบแล้วแต่ไม่มีสิทธิ์ → 403 Forbidden

Login Rate Limit — จำกัดการเข้าสู่ระบบ

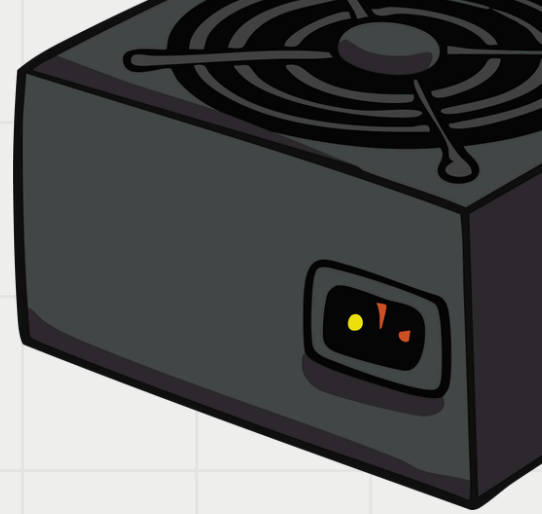
- อนุญาต 5 คำขอต่อ IP ภายใน 60 วินาที
- คำขอที่ 6 → 429 Too Many Requests
- ป้องกันการเดารหัสผ่านแบบ Brute Force





TESTING

Test-Tools



JUnit 5

ทดสอบ Controller และ Service ของ backend
กว่า 40 คลาส ครอบคลุม business logic การ
ยืนยันตัวตน และ security gate

Postman

7 collection ทดสอบ flow
หลักของระบบ: Product
Order Orchestration
Sub-order
Shipping
Review
Seller Application
Payment

Robot Framework

ทดสอบการใช้งานจริงผ่านหน้าเว็บแบบ end-to-end
มี Robot Test ของ User Seller และ Admin
และ full Lifecycle
ทดสอบตั้งแต่ สมัครสมาชิก → ไปเป็นผู้ขาย →
Admin อนุมัติร้านค้า → seller ลงสินค้า → มี user
เข้ามาสั่งซื้อสินค้าชำระเงิน → ผู้ขายจัดส่ง → จนสถานะ
จัดส่ง Complete





Deployment

Frontend
Vercel



Database
Supabase

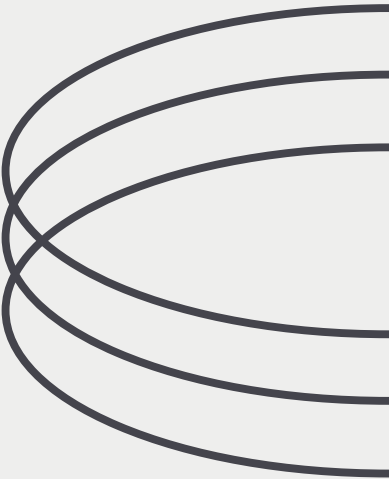


Backend
Railway



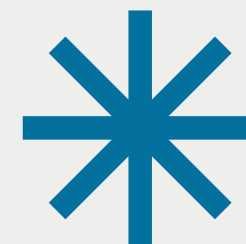
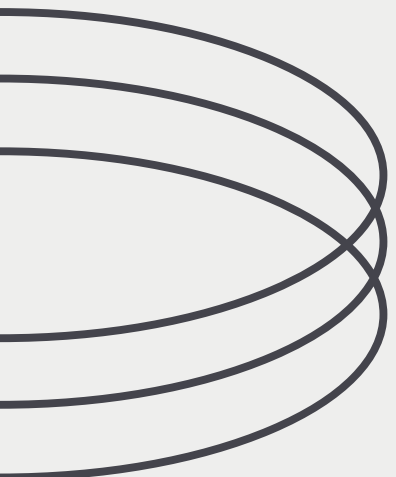
Storage Object
Uploadthing





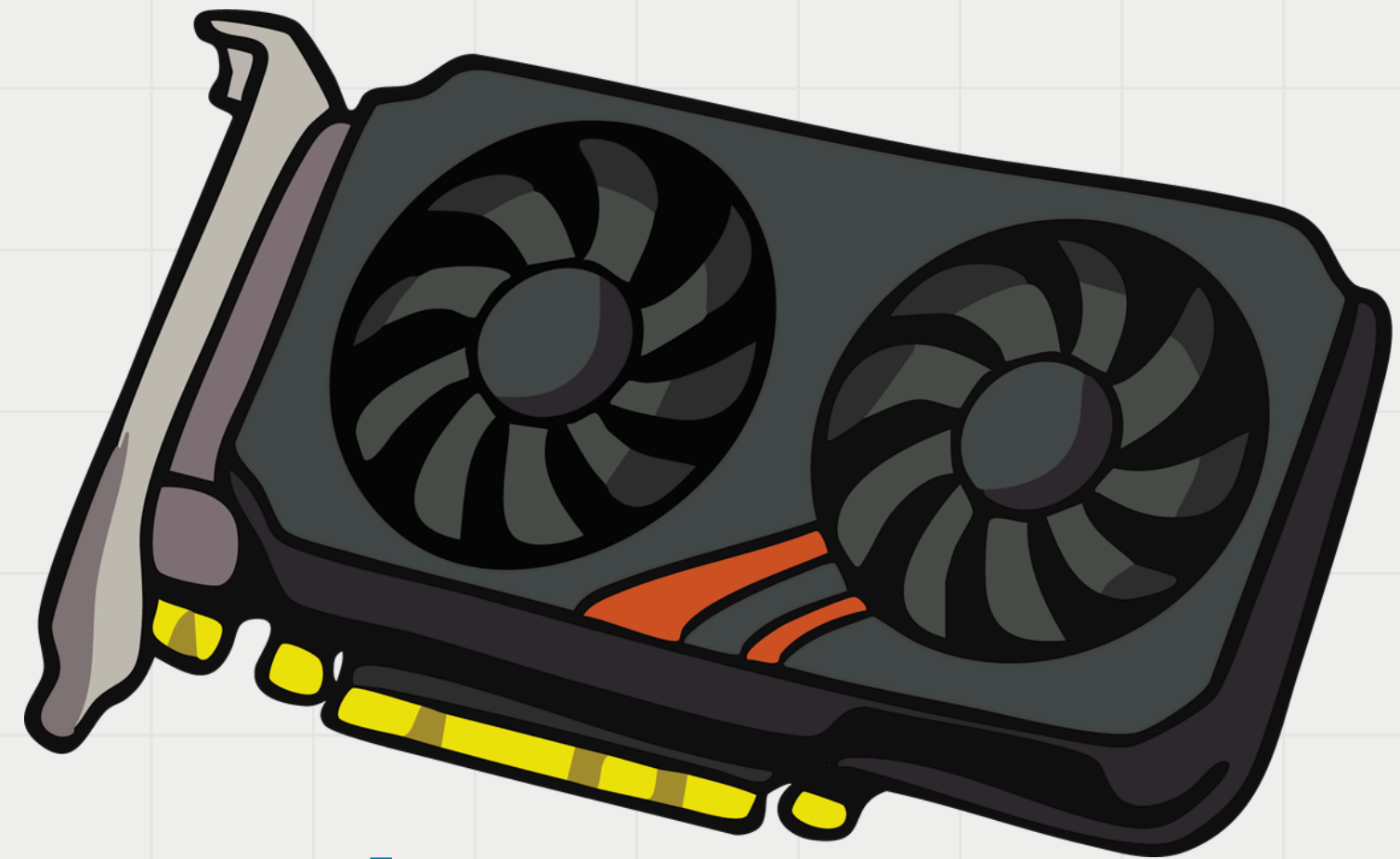
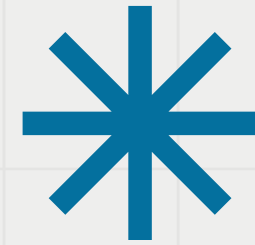
Demo

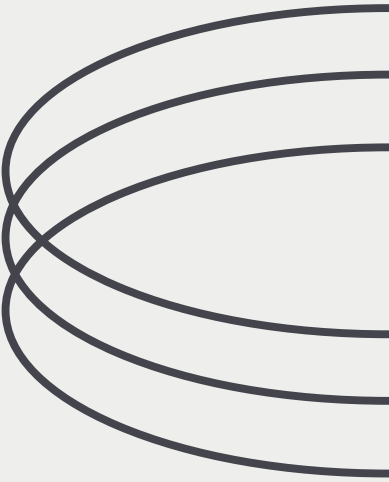
<https://panda-store-kappa.vercel.app/>



ปัญหาที่พบและ แนวทางปรับปรุง

- 1.) ในส่วนของสินค้าทั้งหมดในStockมันจะถูกซ่อนทำให้ User ไม่เห็นสินค้านั้น ข้อเสนอเพิ่มเติมคือควรที่จะทำเป็นสีเทาๆเพื่อให้รู้ว่าร้านค้ายังมีสินค้านี้แต่แค่สินค้าหมดไม่ใช่ไม่มีสินค้า
- 2.) การชำระเงินในระบบของเรายังเป็นแบบ Mock ซึ่งถ้าจะนำไปใช้งานจริงจะต้องต่อPayment GateWay กับทางแพลตฟอร์มการชำระเงินโดยตรงเพื่อตรวจสอบความถูกต้องของการเงิน





**THANK
YOU**

